



STORY ARCHITECT CODEX

User and Maintainer Guide

A practical handbook for the local Codex-powered screenplay editor:
chat, live story context, direct native edits, continuity review, memory,
character safety, build, recovery, and release.

PROJECT-AWARE CHAT	REVIEWED NATIVE EDITS
CONTINUITY + MEMORY	SAFE CHARACTER OPERATIONS

Documentation edition 2026-08-14
Based on Story Architect beta 0.8.3
Local experimental fork - GNU GPLv3

Story Architect Codex is a modified beta build. Save often, leave automatic backups enabled, and keep an independent copy of important projects.

1. What this app is

Story Architect Codex combines the Story Architect writing environment with a project-aware Codex assistant. Story Architect remains the system of record: it owns the screenplay and the native project tabs. Codex reads a request-time story package, reasons with the selected story method, and returns a typed result that STARC validates.

The central design rule is simple: **conversation can be immediate; project changes require review.**

The assistant is intended to behave like a story collaborator inside the editor:

- it can answer questions using the screenplay and linked tabs;
- it can suggest possibilities without modifying anything;
- it can propose native screenplay and story-data edits;
- it preserves screenplay structure during approved generation;
- it remembers useful decisions from older local sessions; and
- it checks continuity before an edit can be applied.

This fork is not an official Story Apps release. The upstream application and this modification are licensed under GPLv3, while some optional plugin modules may have separate access or distribution terms.

2. The app at a glance

The main workspace has three conceptual areas:

Area	Purpose
Project navigator	Opens the project, Characters, Locations, Worlds, screenplay documents, Synopsis, Treatment, Statistics, Recycle Bin, and other available modules.
Native editor	Holds the structured screenplay or selected STARC document. This is where approved direct edits are applied.
AI Assistant panel	Holds the story method, Writer's Room option, chat history, activity state, Story Memory, edit history, prompt field, and review workflow.

Story data is deliberately divided across native tabs:

- **Project metadata** holds high-level information such as the logline.
- **Synopsis** is the compact prose summary.
- **Treatment** is an ordered outline tied to existing treatment paragraphs.
- **Characters** holds native profiles, stable IDs, photos, and relationships.
- **Locations** holds places and associated material; the map is an optional module.
- **Worlds** holds broader worldbuilding records; the map is an optional module.
- **Screenplay** holds formatted scenes and is treated as approved source text.
- **Statistics** analyzes available project/script data when its module is installed.
- **Recycle Bin** keeps removed native records for possible recovery.

Characters, Locations, and Worlds are now kept visible in the project tree even when their collections are empty. Their advanced subviews still depend on the corresponding plugin being present.

3. First-run checklist

Before opening the assistant, confirm the following:

- 1 The local Codex CLI is installed.
- 2 The CLI is authenticated with the OpenAI account you intend to use.
- 3 `codex --version` works in Terminal.
- 4 The story-skills workspace contains `.agents/skills`.
- 5 `STARC_CODEX_WORKSPACE` points to that workspace if it is not at the default location.
- 6 Your `.starc` project has been saved at least once.
- 7 Automatic saving and backups are enabled in STARC settings.

The app locates Codex in this order:

- 1 the executable specified by `STARC_CODEX_BIN`;
- 2 an executable named `codex` on `PATH`;
- 3 `~/local/bin/codex`;
- 4 `/opt/homebrew/bin/codex`; or
- 5 `/usr/local/bin/codex`.

The skills workspace is located in this order:

- 1 `STARC_CODEX_WORKSPACE`;
- 2 `~/Documents/GitHub/AI-Storyboard`; or
- 3 the app's current working directory.

The selected directory must contain `.agents/skills`.

4. Using the chat

Open a screenplay, choose **AI Assistant**, and type into the message field.

- Press **Enter** to send.
- Press **Shift+Enter** to insert a line break.
- Click the send icon for the same action as Enter.
- Click the active cancel control while Codex is working to stop the request.
- Drag across any part of a message to select only that text.
- Use the normal copy shortcut or the message context menu to copy the selection.
- Use **New Chat** to clear the visible conversation.

The panel shows activity such as preparing, starting, connecting, and thinking, with elapsed time while generation is in progress. Long conversations and long assistant messages are vertically scrollable.

Conversational requests

Questions and brainstorming should stay in chat:

```
What is the logline already stored for this screenplay?  
  
What does Lena know by the end of the train-station scene?  
  
Which plot thread has gone quiet?  
  
Give me three ways the next confrontation could turn without changing the script.
```

For these requests, the protocol uses `answer`, `suggest_ideas`, or `request_clarification`; no editor approval is required because nothing is changed.

Direct-edit requests

Use a clear verb and target when you want the app to change native story data:

```
Generate a scene at the end where Lena realizes the witness has been lying.  
  
Replace the selected dialogue with a quieter version that preserves the subtext.  
  
Delete the selected action paragraph.  
  
Clear the entire screenplay.  
  
Update the logline so the moral choice is central.  
  
Create a secondary character named Imani and describe her role in the investigation.
```

Wording matters because an instruction such as "What could happen in the next scene?" asks for ideas, while "Generate the next scene at the end" asks for a project edit.

5. Story methods

The **Story method** selector appears at the top of the AI Assistant panel. The choice is remembered locally.

Story continuity (current)

This mode uses the `edit-story` skill. It is the general-purpose choice for:

- continuing or rewriting scenes;
- preserving the writer's voice;
- tracking character voice and knowledge;
- following chronology and relationships;
- respecting established world rules;
- diagnosing setups and payoffs; and
- answering continuity questions.

Eric Edson method

This mode uses `eric-edson-story-skill`. Choose it when you want analysis or development grounded in:

- Hero Goal sequences;
- three-act tent-pole structure;
- Stunning Surprise One and Stunning Surprise Two;

- hero sympathy and emotional connection;
- adversary, love interest, sidekick, mentor, and endangered innocent functions; or
- act balance and structural diagnosis.

Why the methods are separate

The methods are not silently blended. A visible selection makes the creative lens predictable, keeps the edit record meaningful, and lets the writer compare approaches. Both still receive the same live story package and the same safety protocol.

6. Writer's Room suggestions

Enable **Writer's Room suggestions** when you want Codex to follow your progress while you write.

Writer's Room mode watches meaningful changes made while the screenplay editor has focus. After approximately 45 seconds of quiet, it can add a short advisory note if there was enough activity. It waits at least five minutes before another note.

Each note is limited to roughly 120 words and contains:

- 1 the strongest recent development;
- 2 one continuity or structural watchpoint; and
- 3 one concrete possibility for the next story turn.

It applies the currently selected story method. It does not edit the screenplay and begins with **Room note:** so it is distinguishable from a direct response.

Turn this option off when drafting rapidly, working offline, or conserving Codex usage.

7. What Codex knows on each request

The app builds a live **story package** when the message is sent. The package includes available project information rather than relying only on the current chat.

The package can include:

- title and logline metadata;
- synopsis;
- ordered treatment text;
- the complete screenplay split into scenes;
- character profiles and stable identifiers;
- relationships between characters;
- locations;
- worlds;
- request-time selection and cursor context;
- recent chat memory and older relevant messages; and
- structured Story Memory.

The story package is labeled as source material, not instructions, to reduce prompt injection from text inside a screenplay.

The request also records a story revision number. If the screenplay or linked data changes while Codex is thinking, STARC refuses to apply the old proposal. Send the instruction again from the current draft.

8. Local chat memory

Chat is stored per screenplay document UUID in local Qt settings under keys shaped like:

```
codex/project-memory/<screenplay-uuid>
```

Two related collections are maintained:

- **session** is the visible current conversation;
- **memory** retains up to 2,000 writer and assistant messages for local recall.

For each new request, the app sends the most recent 24 messages plus up to eight older messages that share meaningful terms with the prompt. The retrieved context is capped to prevent an unbounded transcript.

New Chat clears the visible session, not the retained project memory. This allows a later request to recover an earlier decision without showing weeks of messages in the current chat.

Important limitations:

- memory is local to this application settings profile;
- memory is keyed to a screenplay document, not merely its filename;
- memory is not embedded in the `.starc` database;
- moving the project alone to another computer does not move memory; and
- deleting/resetting application preferences can remove it.

9. Structured Story Memory

Chat memory records conversation. **Story Memory** records structured continuity.

Use the Story Memory control to build or refresh an evidence-based record with these sections:

- CHARACTERS & RELATIONSHIPS
- CHARACTER KNOWLEDGE
- TIMELINE
- PLOT THREADS
- SETUPS & PAYOFFS
- WORLD RULES
- VOICE & STYLE
- CONTINUITY RISKS

Codex must distinguish confirmed canon from inference and cite scene headings or linked STARC tabs. Missing facts must not be invented.

The memory proposal is reviewed before saving. The writer can also correct the saved record. When underlying story material changes, the app marks Story Memory stale so it is not mistaken for a current, authoritative snapshot.

Story Memory is stored locally under:

```
codex/story-memory/<screenplay-uuid>
```

It guides later requests but remains working continuity analysis, not automatic canon.

10. STARC Action Protocol V3

For normal story chat, Codex must return exactly one schema-valid Protocol V3 object. The schema prevents the model from mixing prose, an explanation, and an ambiguous editor command.

Every result includes:

Field	Meaning
<code>version</code>	Must be <code>3</code> .
<code>action</code>	One supported conversational or editor action.
<code>target</code>	The exact destination, such as <code>selection</code> , <code>end</code> , <code>logline</code> , or <code>characters</code> .
<code>content</code>	Conversational text or production-ready Fountain/document content.
<code>summary</code>	Short human-readable description.
<code>requiresApproval</code>	<code>true</code> for every editor-changing action.
<code>entityId</code> / <code>entityName</code>	Stable native character identity when relevant.
<code>fieldChanges</code>	Only explicitly proposed structured character/relationship changes.
<code>impactSummary</code>	Concise consequence of the proposed story change.
<code>continuityChecks</code>	Structured critical, caution, or suggestion findings with evidence.

Supported actions

Action	Target	Result
answer	none	Answers a story question in chat.
suggest_ideas	none	Offers possibilities without changing the project.
request_clarification	none	Asks for a safe target or missing decision.
insert_screenplay	cursor, beginning, end	Imports approved Fountain into native screenplay blocks.
replace_selection	selection	Replaces the request-time selection with approved Fountain.
delete_selection	selection	Deletes the request-time selection.
clear_screenplay	none	Clears all screenplay text after explicit confirmation.
update_logline	logline	Replaces the native logline.
replace_synopsis	synopsis	Replaces the native synopsis.
revise_treatment	treatment	Revises existing treatment paragraphs without adding/removing scene slots.
create_character	characters	Creates one native character profile.
update_character	characters	Changes requested fields on one stable character ID.
remove_character	characters	Moves one character profile to Recycle Bin after dependency review.
merge_character	characters	Merges one duplicate into one survivor through a transaction.
update_character_relationship	character_relationships	Creates or updates one native relationship.
update_story_memory	story_memory	Proposes a complete Story Memory refresh.

An unsupported target, missing stable ID, invalid field, malformed Fountain payload, or mismatched approval flag is rejected without modifying the project.

11. Screenplay generation and formatting

For insertion and replacement, Codex returns only production-ready Fountain - no "Here is your scene," no Markdown fence, and no copy/paste wrapper.

STARC parses that Fountain into native paragraph types:

- scene headings;
- action;
- character cues;
- parentheticals;
- dialogue;
- shots;
- transitions; and
- other supported structured screenplay blocks.

Use complete scene headings:

```
INT. KITCHEN - NIGHT
```

```
Rain needles the dark window. MARA grips the forged passport.
```

```
MARA
```

```
(quietly)
```

```
This was never his.
```

Do not ask for headings such as `# Scene 1`; those are outline labels, not screenplay scene headings.

Best generation workflow

For a feature screenplay, work in reviewed units rather than requesting the entire film in one turn:

- 1 lock the premise, logline, characters, and world rules;
- 2 build a treatment or structural plan;
- 3 refresh Story Memory;
- 4 generate one sequence or scene;
- 5 read the entire scrollable proposal;
- 6 inspect the Continuity Gate;
- 7 approve, request a revision, or discard;
- 8 continue from the newly approved canon; and
- 9 periodically refresh Story Memory.

This keeps voice, continuity, format, and author control inspectable.

12. Review, revision, and approval

An editor-changing result opens a review dialog instead of appearing as screenplay prose that must be copied from chat.

The review can show:

- the proposed content or focused before/after diff;
- the destination;
- an edit summary;
- the story-impact summary;
- continuity findings;
- character dependency or merge information when relevant; and
- actions to discard, revise, or apply.

Review content is vertically scrollable. Pressing ordinary keys while reading does not automatically reject the proposal. Use the explicit dialog actions; Escape remains a deliberate cancellation path where enabled.

If you ask Codex to revise the proposal, the original writer instruction, action, target, content, and Continuity Gate are sent back for a new schema-valid result. Nothing is applied until a proposal is accepted.

13. Continuity Gate

Every editor-changing action performs a self-audit against the live screenplay, linked tabs, and Story Memory.

Findings have three severities:

- **critical**: a direct conflict with confirmed canon;
- **caution**: a likely inconsistency, unclear motivation, or risky inference; and
- **suggestion**: an optional improvement.

Evidence should point to a scene heading or linked STARC tab and say when it is inferential.

A critical confirmed-canon conflict requires a second explicit **Approve intentional conflict** decision. This does not claim the assistant is infallible; it creates a visible checkpoint for deliberate retcons.

14. Editing metadata, synopsis, and treatment

Logline

`update_logline` replaces the native logline only after review. Because the live logline is included in the story package, Codex can also quote or analyze it without editing.

Synopsis

`replace_synopsis` replaces the native synopsis after review. The previous and new states are recorded in edit history.

Treatment

`revise_treatment` updates the current treatment outline in place. The returned content must contain exactly one line for every existing editable treatment paragraph, in order. The action does not add or remove treatment scene slots.

This constraint prevents an AI response from silently breaking the treatment's native structure.

15. Characters and relationships

Character actions use stable native UUIDs, never a guessed name alone.

The assistant can create or update these structured fields:

- name, story role, age, and nickname;
- one-sentence and long descriptions;
- family and personality;
- motivation and moral;
- greatest fear and secrets;
- short-term and long-term goals;
- initial and changed beliefs;
- plot involvement and conflict; and
- speech.

Only fields the writer asked to change are sent. An empty value intentionally clears a field. Story role is constrained to primary, secondary, tertiary, or undefined.

Relationship updates identify the other character by stable ID and may change:

- feeling; and
- details.

The protocol does not currently support deleting a relationship.

16. Safe character removal

Removing a character does not permanently delete it. The app first builds a dependency report containing:

- name mentions in the current screenplay;
- dialogue counts across project scripts;
- synopsis and treatment mentions;
- Story Memory mentions;
- incoming and outgoing relationships; and
- attached photo count.

After approval, only the native character profile moves to Recycle Bin. Screenplay prose, synopsis, treatment, Story Memory text, and stored relationship links are preserved. This avoids destructive search-and-replace behavior and allows the profile to be restored through STARC.

17. Transaction-safe character merging

Character merge exists for true duplicates that should become one native identity.

Merge plan

The proposal identifies:

- the **survivor** whose stable ID remains;
- the **duplicate** that will move to Recycle Bin;
- the final value for every meaningful conflicting profile field;
- photos to transfer;
- outgoing relationships to transfer;
- incoming relationships to reassign; and
- affected native script cues.

If both characters have different non-empty field values and the writer has not supplied a precedence or safe combination, Codex must ask for clarification rather than guess.

What changes

After approval, STARC:

- updates the survivor's resolved profile fields;
- transfers unique photos;
- combines compatible relationship details;
- redirects incoming and outgoing relationship links;
- reassigns native character cues in screenplay, comic-book, audioplay, and stageplay documents;

- leaves ordinary action/dialogue prose untouched; and
- moves the duplicate's complete original profile to Recycle Bin.

Transaction journal and rollback

Before the merge, the app snapshots every affected character and script document. Journals are stored locally under keys shaped like:

```
codex/character-merge/transactions/<transaction-id>  
codex/character-merge/latest/<duplicate-character-id>
```

Edit history can restore the complete pre-merge transaction: survivor, duplicate, photos, relationships, and native cues. Because rollback restores snapshots, it warns that later edits in affected documents can be replaced. If the project no longer matches the expected transaction state, rollback refuses rather than applying a partial restoration.

18. Codex edit history

The app keeps the latest 100 Codex-applied changes per screenplay under:

```
codex/edit-history/<screenplay-uuid>
```

Each entry records:

- UTC timestamp;
- action;
- writer instruction;
- summary;
- target;
- selected story method;
- before and after state;
- impact summary; and
- continuity findings.

The history viewer is newest-first and supports older/newer navigation. Where safe, it can restore earlier screenplay, logline, synopsis, treatment, character, or relationship state. Character creation/removal have different recovery semantics; full transaction-aware merge rollback is offered only when its journal is available and committed.

The editor's normal undo stack and Codex edit history serve different purposes: normal undo is immediate document editing, while Codex history is a persistent local audit trail across sessions.

19. Saving and recovery

Story Architect stores the project in a user-selected `.starc` file. The format is a local project database containing the screenplay and native project documents.

Default storage behavior in this source tree:

- auto-save: enabled;
- backups: enabled;
- default backup folder: `~/Documents/starc/backups`;

- retained backups: 7.

Use **Save As** or copy the `.starc` file when creating a milestone. Before major AI-assisted structural edits, an additional manual backup is prudent even though proposals are reviewed and history is recorded.

Do not confuse project data and assistant state:

Data	Stored in <code>.starc</code>	Stored in local settings
Screenplay and native tabs	Yes	No
Chat session and long-term message memory	No	Yes
Structured Story Memory	No	Yes
Codex edit history	No	Yes
Character merge transaction journals	No	Yes
Selected story method / Writer's Room toggle	No	Yes

20. Privacy, usage, and sensitive data

The app launches the local `codex app-server` process. It does not use the ChatGPT website or STARC's legacy hosted AI service for this integration.

The authenticated Codex CLI determines account access and usage. The old STARC **words available / purchase** label remains from the upstream interface and should not be treated as the Codex usage meter.

When you send a request, the live story package and relevant memory context are passed to the local Codex process, which communicates with OpenAI under that CLI's authentication. Consider screenplay content, character data, chat, continuity memory, edit history, and logs sensitive.

The repository ignores:

- `.starc`, Fountain, Final Draft, Trelby, and Celtx project/script files;
- `.env` files and common credential/key formats;
- local Codex and Claude state directories;
- logs, backups, and crash reports;
- Qt/qmake build products;
- app bundles and debug symbols; and
- local/prebuilt plugin binaries.

Before a commit, still inspect `git status` and staged diffs. `.gitignore` does not protect a sensitive file that was already tracked.

21. Optional modules and tabs

The public upstream tree references advanced feature modules through Git submodules. Some may be inaccessible or absent from a normal public clone. These include implementations for:

- character information, dialogues, and relations;

- location information, scenes, and map;
- world information and map;
- screenplay statistics, cards, timeline, breakdown, and series planning;
- novel cards/statistics/timeline;
- presentation and mind map; and
- additional format-specific statistics.

The local app may load compatible prebuilt plugins for these modules. Those binaries are intentionally excluded from Git because they are machine-specific and may be governed by separate terms.

The source contains compatibility handling for older prebuilt plugin interfaces, but binary compatibility is never guaranteed across Qt or compiler changes. A missing plugin should make the module unavailable; it should not justify publishing a private binary.

22. Build guide for macOS

Requirements

- macOS development tools / Clang;
- qmake and a consistent Qt installation;
- C++17 toolchain;
- Git submodules available to your account; and
- Codex CLI for assistant functionality.

Source build

```
git clone <your-fork-url> starc
cd starc
git submodule update --init --recursive
cd src
qmake starc.pro
make -j4
```

Expected bundle:

```
src/_build/starcapp.app
```

Deploy Qt only after the final link

```
macdeployqt src/_build/starcapp.app -always-overwrite
codesign --force --deep --sign - src/_build/starcapp.app
```

If you relink [Libcorelib](#) or any plugin after deployment, deploy and sign again.

Iterative local installation

The canonical repository includes a developer shortcut for updating an existing compatible local installation without replacing all of its optional plugins:

```
scripts/install-local-codex-build-macos.sh \
src/_build/starcapp.app \
"/Applications/Story Architect Codex.app"
```

The script copies the rebuilt core library, core plugin, and screenplay plugin into the installed bundle. It then rewrites absolute Qt framework dependencies to the app's bundled frameworks, refuses any remaining non-bundled Qt reference, ad-hoc signs the complete app, and verifies the signature.

The destination app must already exist and be binary-compatible with the build. Back up the installed app first. Use a full clean deployment when Qt, ABI, or optional-plugin compatibility is uncertain.

Dependency-path verification

```
otool -L src/_build/starcapp.app/Contents/MacOS/starcapp
otool -L src/_build/starcapp.app/Contents/Frameworks/libcorelib.dylib
find src/_build/starcapp.app/Contents/PlugIns -name '*.dylib' -print
```

Inspect important plugins with `otool -L`. The app executable, core library, and plugins must agree on the bundled Qt frameworks. A previous startup crash occurred because a newly linked core plugin referenced Homebrew Qt while the executable loaded bundled Qt. The failure appeared during font database initialization, but the real cause was a mixed Qt runtime.

Signing

Ad-hoc signing is suitable for a local test bundle:

```
codesign --force --deep --sign - src/_build/starcapp.app
codesign --verify --deep --strict --verbose=2 src/_build/starcapp.app
```

Public distribution requires an appropriate Developer ID, hardened runtime/notarization decisions, and a license review for every bundled dependency and optional plugin.

23. Maintainer architecture notes

The integration spans four main layers:

- 1 **AI Assistant UI** - chat, story method, Writer's Room, activity, memory/history controls, and selection/copy behavior.
- 2 **Screenplay manager/view** - builds the live story package, captures request-time context, validates Protocol V3, shows reviews, applies native edits, records history, and guards revisions.
- 3 **Codex service manager** - finds the CLI, launches `codex app-server`, initializes it, selects the story skill, defines the output schema, queues tasks, and reports activity/errors.
- 4 **Project manager/models** - performs native project operations, Recycle Bin movement, relationship work, multi-document cue reassignment, merge journaling, and rollback.

The selected story skill is stored at:

```
codex/story-assistant/skill
```

Writer's Room preference is stored at:

```
codex/story-assistant/writers-room-enabled
```

Do not weaken these invariants when extending the protocol:

- one response equals one action;
- conversational actions never require or cause editor changes;
- editor actions always require approval;
- stable native IDs are mandatory for existing entities;
- request-time state is checked again before apply;
- malformed/unsupported actions fail closed;
- screenplay content is imported as structured blocks, not pasted as presentation prose;
- project-wide changes have a dependency report or transaction snapshot; and

- every applied Codex edit is auditable.

24. Verification checklist

Run this checklist before treating a build as usable.

Launch and project navigation

- Launch without a project.
- Create and save a blank project.
- Open an existing `.starc` project.
- Reopen the same project after quitting.
- Open Screenplay, Synopsis, Treatment, Characters, Locations, Worlds, Statistics, and Recycle Bin.
- Open Relations and Location Map when their plugins are present.
- Confirm empty Characters/Locations/Worlds containers remain visible.

Chat

- Enter sends; Shift+Enter creates a new line.
- The send icon works.
- Activity changes from preparing/connecting to thinking.
- Cancel stops an active request.
- Long chat scrolls.
- A partial message selection can be copied.
- New Chat clears the visible session.
- Reloading the same screenplay restores its session/memory behavior.

Context and methods

- Ask for the existing logline and compare the answer to metadata.
- Ask about a character, relationship, location, and treatment detail.
- Switch between Story continuity and Eric Edson, then verify the saved choice.
- Enable Writer's Room, make a meaningful edit, wait for the room note, then disable it.
- Build, inspect, correct, and refresh Story Memory.

Direct edits and safety

- Generate one short scene and scroll through the entire review.
- Discard it and confirm the screenplay is unchanged.
- Generate again and approve; confirm native formatting.
- Replace a selection and confirm only the request-time selection changes.
- Start a request, edit the story before it returns, and confirm stale protection.
- Trigger a continuity caution.
- Trigger a test critical conflict and confirm second approval is required.

- Update logline, synopsis, and treatment; inspect edit history.
- Restore an earlier supported version.

Characters

- Create a character with structured fields.
- Update one field without changing unrequested fields.
- Add/update a relationship using stable IDs.
- Request removal, inspect the dependency report, and verify Recycle Bin behavior.
- Merge two disposable test characters with conflicting data.
- Verify photos, relationships, and native cues transfer.
- Roll back the merge and compare all affected documents to pre-merge state.

Packaging

- Confirm generated files are ignored.
- Confirm no `.starc`, script exports, secrets, logs, or crash reports are staged.
- Confirm no private/prebuilt plugins are staged.
- Run `macdeployqt` after the last link.
- Verify Mach-O dependency paths.
- Verify the final code signature.
- Launch the final copied/installed bundle, not only the build-tree bundle.

25. Troubleshooting

The app crashes immediately

- 1 Open the newest `starcapp-*.ips` report in `~/Library/Logs/DiagnosticReports`.
- 2 Check every core Mach-O dependency with `otool -L`.
- 3 Ensure no plugin brings in a different Qt runtime.
- 4 Re-run `macdeployqt` after the final link.
- 5 Re-sign the complete bundle.
- 6 Temporarily remove optional plugins to isolate a binary compatibility problem.

The app opens but a character/location/statistics sub-tab is missing

The project data may still be present. Check whether the corresponding optional plugin exists and loaded. Missing views are not proof that character or location records were deleted.

Codex does not start

Run:

```
codex --version
which codex
```

If the GUI environment cannot see your shell path, set `STARC_CODEX_BIN` to the absolute executable path before launching the app.

A story method cannot be used

Confirm:

```
test -f "$STARC_CODEX_WORKSPACE/.agents/skills/edit-story/SKILL.md"
test -f "$STARC_CODEX_WORKSPACE/.agents/skills/eric-edson-story-skill/SKILL.md"
```

Use a real absolute path for `STARC_CODEX_WORKSPACE`.

Codex answered in chat instead of editing

Use an explicit edit verb, provide the target, or create a selection first. Compare:

```
Could this scene be shorter?
```

with:

```
Replace the selected scene with a shorter version that keeps the reveal.
```

The review will not apply

Read the displayed reason. Common causes are:

- the response is malformed or unsupported;
- the required selection no longer exists;
- the story changed while Codex was thinking;
- the character stable ID no longer resolves;
- a merge plan did not resolve every conflicting field; or
- a critical conflict needs additional approval.

Chat history disappeared

Check that you opened the same screenplay document and did not reset preferences. Chat is keyed by screenplay UUID. A copied/imported document may receive a different identity.

26. Recommended next improvements

The current foundation is usable for real testing. The next high-value work should favor reliability over adding many new actions:

- 1 Export/import assistant memory with the project, using an encrypted or writer-controlled sidecar.
- 2 Add a first-class usage/status view sourced from the Codex CLI instead of the legacy word counter.
- 3 Add automated Protocol V3 validation and native-edit integration tests.
- 4 Add a preflight screen listing missing/incompatible optional plugins before a project opens.
- 5 Add project-level privacy controls for which tabs are included in a Codex request.
- 6 Add named conversation sessions instead of one visible chat plus background memory.
- 7 Add an edit-plan mode for multi-scene changes before generation begins.
- 8 Add portable, versioned Story Memory with migration and conflict handling.
- 9 Add crash-safe recovery tests for character merge and project-wide edits.

10 Replace legacy STARC AI purchase UI when the Codex integration is active.

27. Glossary

Term	Meaning
Canon	Approved story facts currently present in the screenplay and linked STARC tabs.
Codex CLI	The locally installed OpenAI coding-agent command used through <code>codex app-server</code> .
Continuity Gate	Structured story-impact and conflict review attached to editor changes.
Direct edit	A reviewed Protocol V3 action applied to a native STARC document.
Fountain	Plain-text screenplay syntax used as the structured interchange format for generated script content.
Project memory	Persistent local writer/assistant messages keyed to one screenplay UUID.
Request-time snapshot	The selection, cursor context, story package, and revision captured when a prompt is sent.
Stable ID	Native UUID identifying an existing character independent of its name.
Story Memory	Evidence-based structured continuity map stored locally for one screenplay.
Story method	The selected story-analysis skill, currently Story continuity or Eric Edson.
Transaction journal	Pre-change snapshots used to roll back a character merge as one project-wide operation.
Writer's Room	Optional advisory notes produced after meaningful writing and a quiet period.

28. Credits and license

Story Architect was created by Story Apps, the team behind KIT Scenarist. This modified fork adds the local Codex integration and related workflow, context, editing, memory, continuity, UI, and stability changes described here.

The repository retains the GNU General Public License version 3. Modified builds must be marked as modified and distributed in compliance with GPLv3. Optional plugins, third-party dependencies, trademarks, and external services may have additional terms; verify them before redistribution.

Official upstream resources:

- <https://github.com/story-apps/starc>
- <https://starc.app/>

The printable edition is generated from this file with ReportLab:

```
python3 -m pip install reportlab
python3 docs/generate_pdf.py
```